

SHL Conversational Assessment Recommender

Approach Document

Dhuruv Kumar Gupta

1 Problem Overview

Hiring managers often begin with vague requirements such as:

“I am hiring a Java developer”

However, they may not know which assessments are most suitable for evaluating candidates. Traditional catalog search systems rely heavily on keywords and filters, assuming users already know the appropriate assessment terminology.

The goal of this assignment was to build a conversational recommendation system that can guide users from vague hiring requirements toward suitable SHL assessments through dialogue.

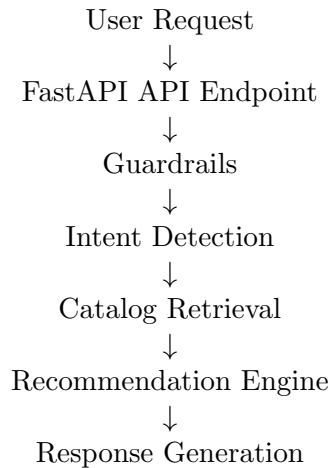
The system supports:

- Clarification for vague queries
- Recommendation of suitable assessments
- Refinement of recommendations
- Assessment comparison
- Off-topic refusal behavior

The system remains restricted to SHL catalog content and avoids generating unsupported recommendations.

2 System Architecture

The project follows a lightweight stateless API architecture using FastAPI.



The application is stateless. Each request contains the complete conversation history and no server-side session memory is stored.

3 Retrieval Approach

The SHL product catalog was used as the primary knowledge source.

Catalog entries are loaded from:

`shl_catalog.json`

Each catalog item contains information such as:

- Assessment name
- Description
- Test type
- URL

To make retrieval easier, fields are merged into a searchable text representation.

Example:

`name + description + category + skills`

Initially, embedding-based retrieval using Sentence Transformers and FAISS was explored. However, deployment memory limitations on the hosting platform created resource issues.

To maintain simplicity and deployment reliability, retrieval was implemented using lightweight text similarity matching with Python's SequenceMatcher.

The system computes similarity scores between user queries and catalog entries and returns the highest ranked matches.

4 Conversation Logic

The application handles multiple conversational behaviors required by the assignment.

Clarification

If the user input is vague:

`Need assessment`

the system requests more information before recommending.

Example:

`Could you share role, skills,
or seniority level?`

Recommendation

When enough information is available, relevant assessments are retrieved and returned.

Refinement

The API is stateless; therefore the entire conversation history is passed with each request.

User messages are merged into a unified context:

`Hiring Java developer
Actually add personality tests`

This enables recommendation refinement without restarting the conversation.

Comparison

Queries such as:

`Compare OPQ vs GSA`

trigger a comparison flow that retrieves catalog-based assessment information.

5 Guardrails

The system was restricted to SHL-related interactions.

Guardrails were implemented to:

- Reject unrelated requests
- Refuse prompt injection attempts
- Block legal and non-SHL advice
- Ensure recommendations originate only from catalog data

Examples:

`Ignore previous instructions`

`Give stock advice`

Such requests are rejected with a controlled response.

6 Testing and Evaluation

Multiple conversational scenarios were tested manually:

- Vague queries
- Recommendation generation
- Refinement behavior
- Assessment comparison
- Off-topic prompts
- Schema validation

Swagger documentation and deployed endpoints were used during testing.

7 Technology Stack

The following tools and frameworks were used:

- Python
- FastAPI
- Pydantic
- Render
- SHL Catalog JSON
- SequenceMatcher

8 Deployment

The project was deployed publicly using Render.

API endpoints:

`/health`

`/chat`

The deployed service supports evaluation through public HTTP access.